

TDT4195 Exercise 1

Kacper Krzysztof Maciejko

Clément Jourdin

September 3, 2026

1 Task 1: Drawing your first triangle

1.1 (c)

Define and instantiate a VAO containing at least 5 distinct triangles using the function you defined in (a). Use the shader pair you loaded in (b) to draw the VAO elements.

```
let vertices_vec_4: Vec<f32> = vec![
    -0.6, -0.6, 0.0, 1.0, 1.0, 1.0,
    0.6, -0.6, 0.0, 1.0, 1.0, 1.0,
    0.0, 0.6, 0.0, 1.0, 1.0, 1.0,

    -0.6, 0.6, 0.0, 1.0, 1.0, 1.0,
    0.6, 0.6, 0.0, 1.0, 1.0, 1.0,
    0.0, 0.9, 0.0, 1.0, 1.0, 1.0,

    -0.8, -0.8, 0.0, 1.0, 0.0, 0.0,
    -0.4, -0.8, 0.0, 1.0, 0.0, 0.0,
    -0.6, -0.65, 0.0, 1.0, 0.0, 0.0,

    -0.5, 0.4, 0.0, 1.0, 1.0, 1.0,
    0.5, 0.4, 0.0, 1.0, 1.0, 1.0,
];

// adding more triangles, we also have to add more indices (3 for each)
let indices_vec_4: Vec<u32> = vec![
    0, 1, 2,
    3, 4, 5,
    6, 7, 8,
    0, 9, 3,
    1, 10, 2,
];
```

1.2 Task 2: Geometry and Theory

1.2.1 a)

The triangle is truncated on 2 sides because 2 of its vertices have a coordinate which is outside of the coordinate system we are working on (vertices 0 and 2 have respectively -1.2 and 1.2 as their z coordinate, and the coordinate system ranges from -1 to 1).

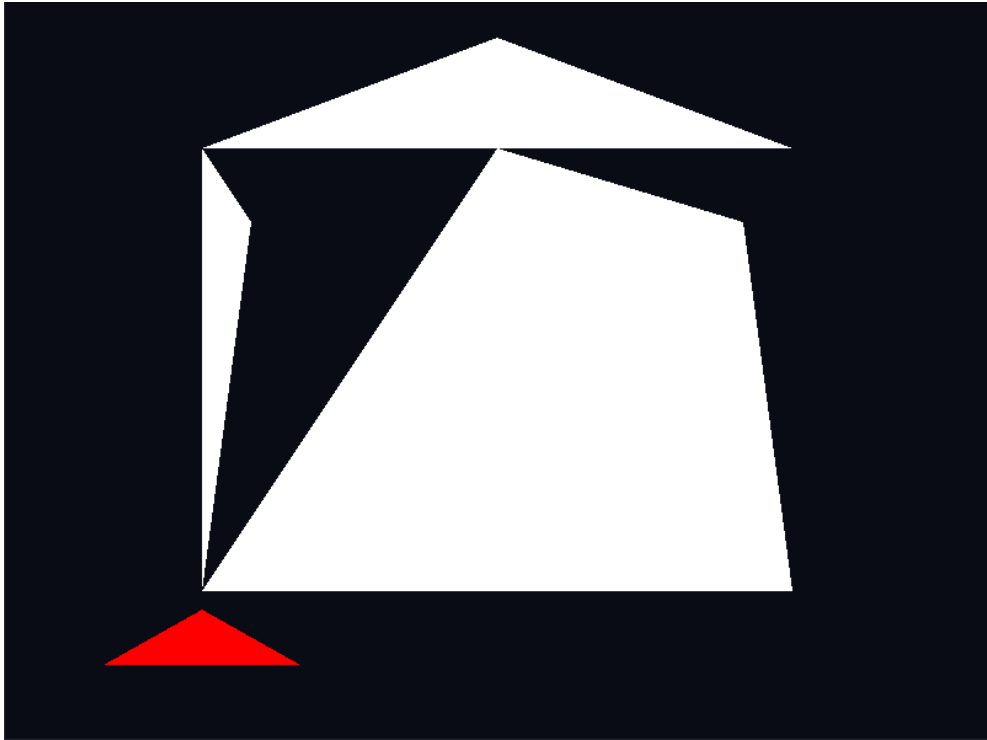


Figure 1: 5 distinct triangles

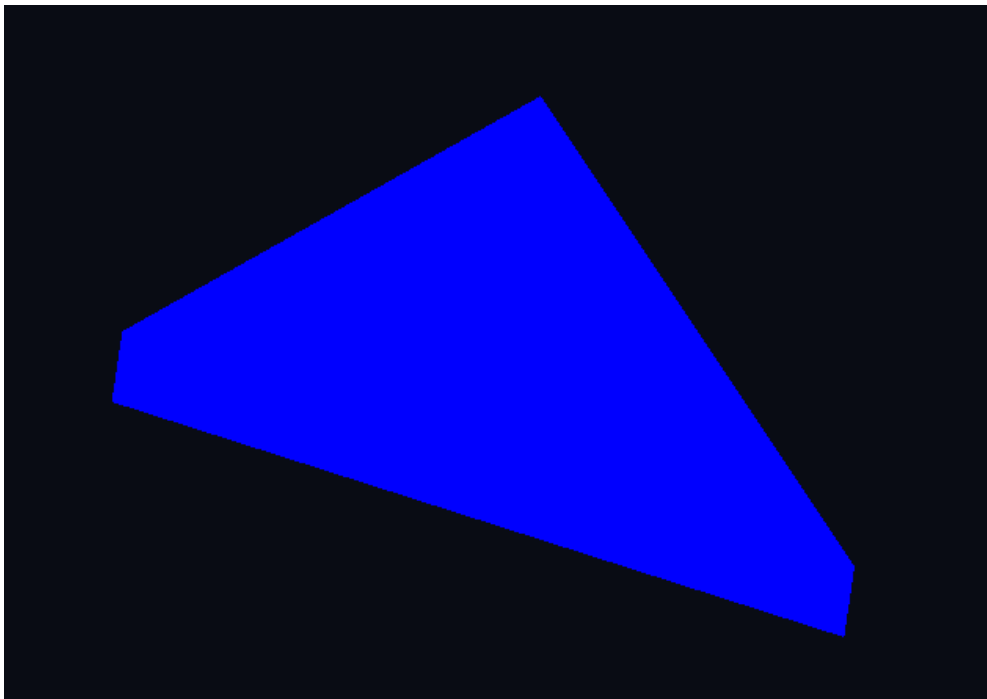


Figure 2: A triangle experiencing clipping

This phenomenon is called **clipping** and it ensures that images behind the camera are not visible in the rendered image.

1.2.2 b)

```
let vertices2_vec_4: Vec<f32> =  
    vec![  
        0.6, -0.8, -1.2, 0.0, 0.0, 1.0,  
        0.0, 0.4, 0.0, 0.0, 0.0, 1.0,  
        -0.8, -0.2, 1.2, 0.0, 0.0, 1.0,  
    ];  
  
let indices2_vec_4: Vec<u32> = vec![  
    0, 2, 1,  
];
```

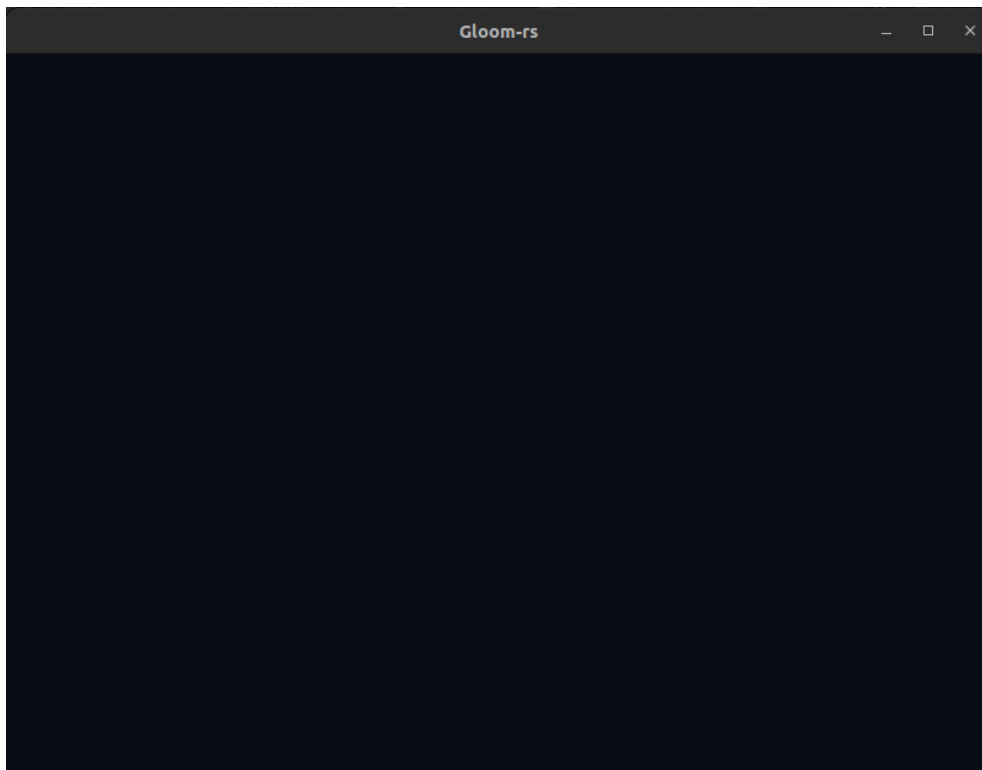


Figure 3: No triangle

When we put the vertices clockwise instead of counterclockwise in the index buffer, no triangle is displayed on the screen. This can be explained by the fact that the `gl::DrawElements` function supplies the points in counter-clockwise order. Therefore, if the points we give to it are in a clockwise order, the back face of the triangle is rendered on the screen.

1.2.3 c) Geometry and Theory

1.2.3.1 i) The depth buffer holds the depth of each pixel in the scene. Each fragment is compared against the depth buffer's value at that point. If a scene changes, but the depth buffer is not cleared, the pixels in a new scene will be compared against the depth values of the previous frame. That means that a sphere moving rightward would leave a trail of hidden background pixels in its wake, because their depth would still be registering as being hidden behind an object - even if said object has since moved.

1.2.3.2 ii) The Fragment Shader can be executed multiple times for the same pixel when several points share the same x and y coordinates but don't have the same depth (z coordinate). Only the point that is most on top of the scene (has the bigger z coordinate) will be shown on the finale scene, but the Fragment Shader is still called for every point.

1.2.3.3 iii) The two most commonly used types of Shaders are: * Vertex Shader - shader responsible for transforming individual vertices around a scene, as well as projecting the scene onto the camera. * Fragment Shader - shader responsible for determining the colour of each fragment

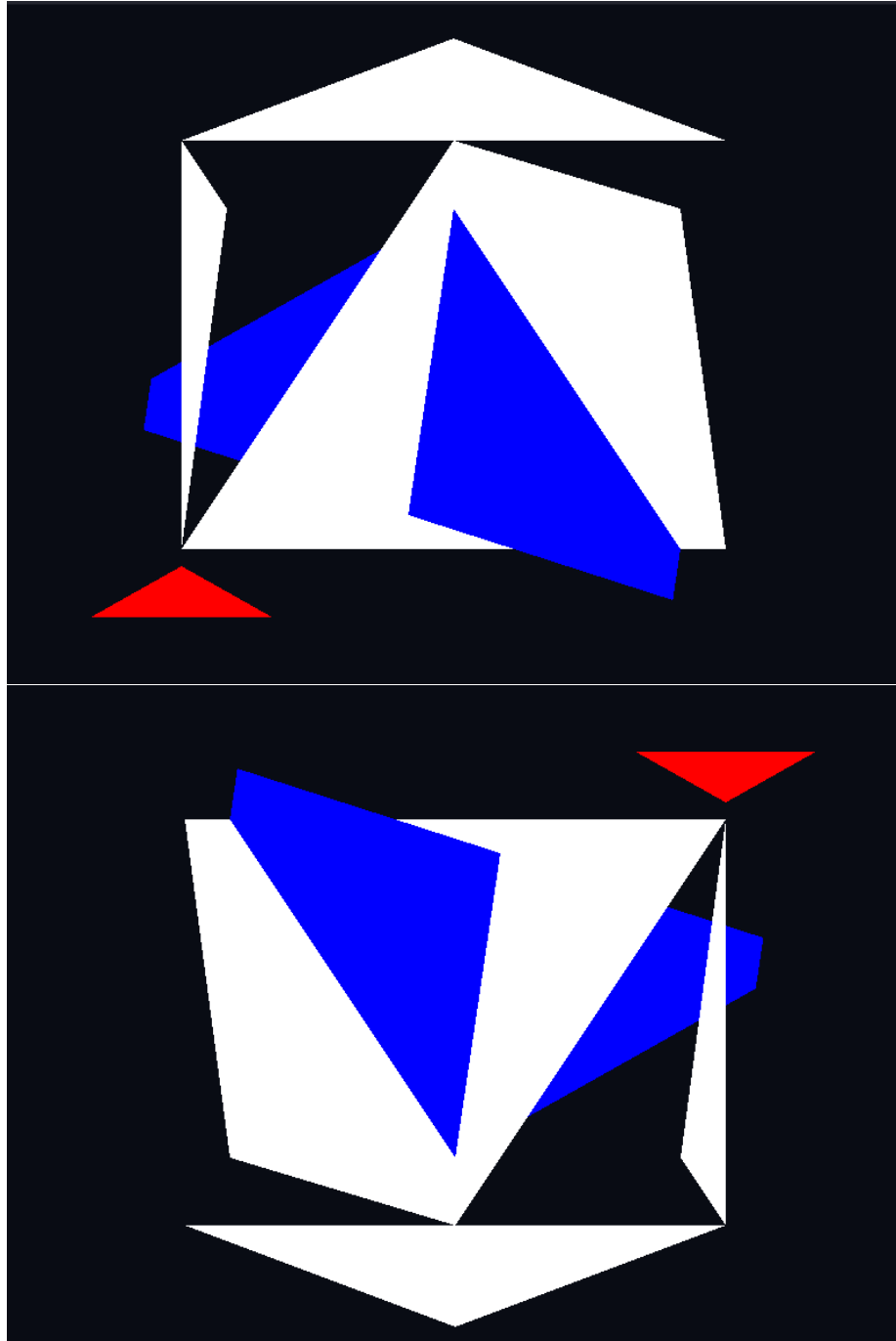
1.2.3.4 iv) The practice comes from the fact that vertices can be used multiple times over different triangles. Using an index buffer enables us to define the vertices once, and combining them together by using their index in the VBO. This can save memory usage, if executed correctly.

1.2.3.5 v) Using multiple entry types (for example passing a color of vertex alongside its position) would cause passing a non-zero value into this function. This is shown in the example below.

```
gl::VertexAttribPointer(  
    1,  
    3,  
    gl::FLOAT,  
    gl::FALSE,  
    (6 * sizeof::<f32>()) as gl::types::GLint,  
    (3 * sizeof::<f32>()) as *const gl::types::GLvoid,  
);
```

1.2.4 d)

1. Mirror/flip the whole scene both horizontally and vertically at the same time



Flipping of the scene was achieved by multiplying the position vector by a transformation matrix (scaling the x and y axis by -1), as shown in the code below:

```
#version 430 core
```

```

layout (location = 0) in vec3 position;
layout (location = 1) in vec3 color;

out VS_OUTPUT {
    vec3 color;
} OUT;

void main()
{
    mat3x3 matrix = {{-1.0, 0.0, 0.0}, {0.0, -1.0, 0.0}, {0.0, 0.0, 1.0}};
    vec3 newPosition = matrix*position;
    gl_Position = vec4(newPosition, 1.0f);
    OUT.color = color;
}

```

2. Change the colour of the drawn triangle(s) to a different colour The result of the colour change is visible in the images detailing the flipping part of the task already. We have achieved this, by modifying the Vertex Shader to not only take the position of vertex as argument, but its colour as well. Each vertex now has six floating point values describing it, instead of three: the first three being responsible for its position, and the last three for its colour in the rgb model. The vertex shader then passes the colours to the fragment shader:

```

// Vertex Shader
#version 430 core

layout (location = 0) in vec3 position;
layout (location = 1) in vec3 color;

out VS_OUTPUT {
    vec3 color;
} OUT;

void main()
{
    mat3x3 matrix = {{-1.0, 0.0, 0.0}, {0.0, -1.0, 0.0}, {0.0, 0.0, 1.0}};
    vec3 newPosition = matrix*position;
    gl_Position = vec4(newPosition, 1.0f);
    OUT.color = color;
}

// Fragment Shader
#version 430 core

in VS_OUTPUT {
    vec3 color;
} IN;

out vec4 color;

void main()
{

```

```
    color = vec4(IN.color, 1.0f);  
}
```

Modifying the colour of all vertices in the scene could also be done in an easier way, by simply modifying the colour in the Fragment Shader itself:

```
#version 430 core  
  
out vec4 color;  
  
void main()  
{  
    color = vec4(1.0f, 1.0f, 1.0f, 1.0f); // modifying colour here would result in  
    // change of colour for all triangles in scene  
}
```